✿ MCP SIDEBAR · GENTLE SHELL / PI

# MCP Sidebar: el panel que no sabías que necesitabas

Una idea simple: ver tus servers MCP sin abrir comandos, leer sus tools sin salir del flujo y conservar el foco del editor.

- Diseñado, construido, verificado y entregado paso a paso — la historia de ingeniería completa está en estas slides.
- Lucas M. Sastre · para la comunidad de Gentle Shell y Pi.

✿ EL PROBLEMA

# MCP estaba conectado, pero no se veía

`/mcp` es autoritativo para conexiones y auth, pero no es una vista: cada consulta interrumpe el flujo de trabajo.

Las tools registradas viven en un menú: ver descripciones y capacidades exige entrar y salir del contexto.

Sin una vista persistente, no hay memoria visual de qué hay disponible en esta sesión.

El editor conserva el foco — cualquier solución debía ser pasiva y no capturante.

- Observado en sesiones reales de Gentle Shell con hasta 8 namespaces MCP activos.
- Restricción declarada desde el inicio: `/mcp` sigue siendo la interfaz autoritativa de conexión y autenticación.

✿ LAS REGLAS DEL JUEGO

# No tocar el host, no robar el foco, no mentir

Pasivo por contrato: `nonCapturing: true` — nunca secuestrar el foco del editor.

Honesto por diseño: decir `tools registered` y delegar el estado de conexión a `/mcp`. Nunca inventar un estado conectado.

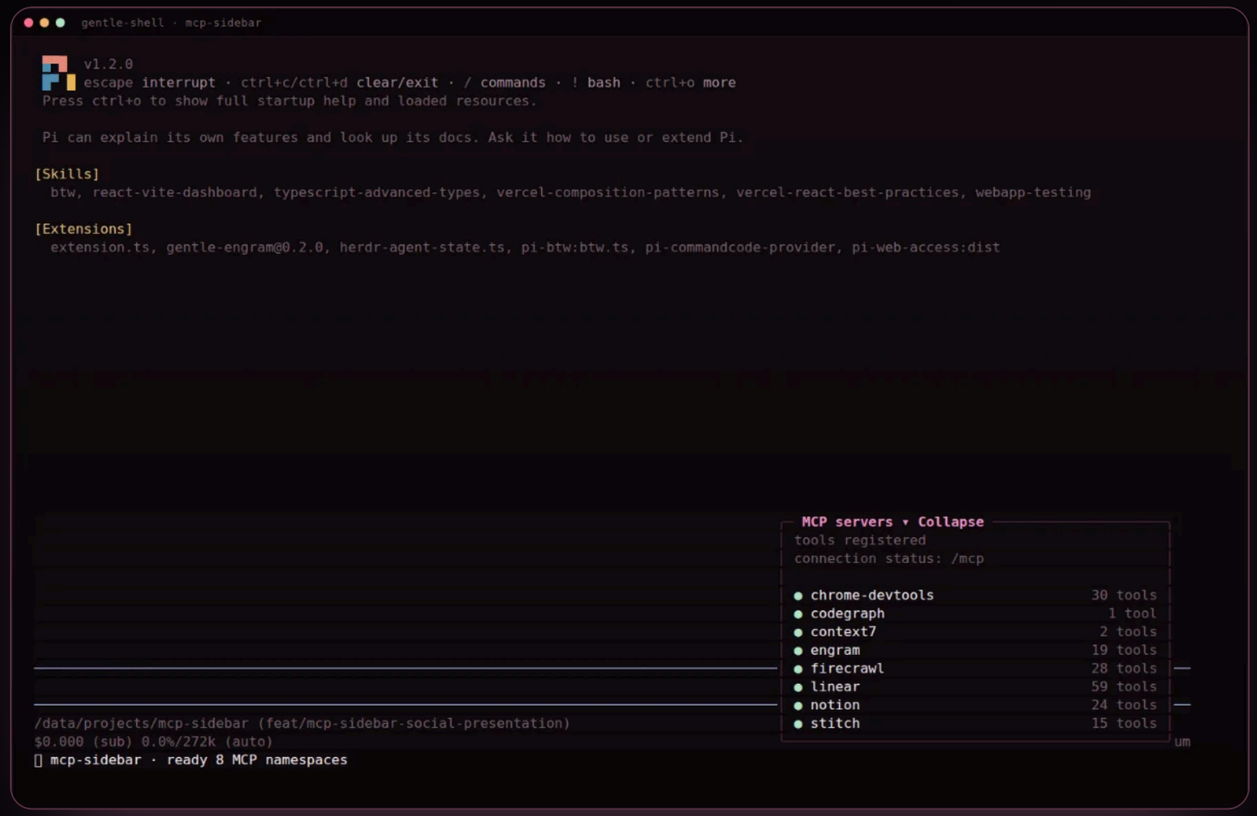
Compatibilidad sobre invasión: el sidebar se integra con el rail nativo mediante feature detection — Gentle Shell queda intacto.

- El copy de estado se corrigió tras una revisión: `status unknown` se reemplazó por `tools registered` + `connection status: /mcp`.
- Ninguna parte de Gentle Shell fue modificada; la integración vive 100% en mcp-sidebar.

✿ LA EVOLUCIÓN

# De overlay flotante a rail nativo

- Primera versión: overlay pasivo abajo a la derecha, alineado a las medidas del rail `Status` de Gentle Shell.
- El feedback del usuario fue claro: el espacio entre paneles era ruido visual — MCP debía fluir dentro del rail.
- La solución: un adapter feature-detected que compone el footer existente por identidad y apila paneles por altura real.
- Falla segura: si el rail desaparece o cambia de dueño, el overlay standby retoma sin romper nada.

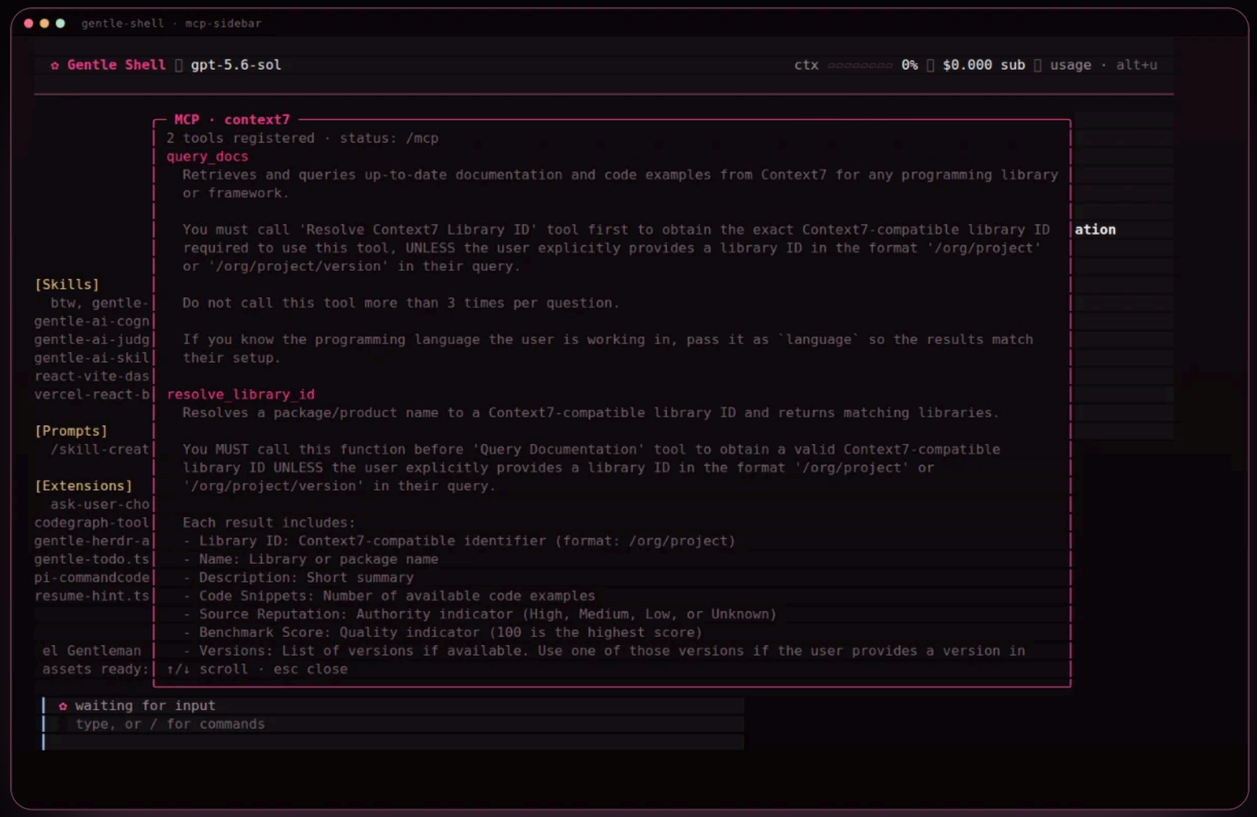



- Overlay inicial: 48 columnas, ancla bottom-right, margen derecho 2 — medidas tomadas del rail `Status` real.
- El adapter detecta `Symbol.for("gentle-pi.experimental-sidebar.state")` y compone el footer por identidad, restaurándolo solo si sigue siendo dueño.

✿ INTERACCIÓN

# Un click en el namespace, toda la info de sus tools

- Click en una fila visible del sidebar abre el detalle de ese namespace — sin robar el foco.
- En modo regular o sin mouse: `/mcp-tools` lista los namespaces, `/mcp-tools context7` abre uno directo.
- El modal es scrolleable, temático y acotado: `↑` `↓`, `Page Up` `Page Down`, `Escape` o `Ctrl+C` para volver.
- Las descripciones vienen de la tool real registrada — sin inventar ni resumir de más.




▸ Verificado en vivo con Gentle Shell 140×40: click real (SGR) sobre `context7` abre sus 2 tools con descripciones completas.

Cuerdas de ciclo de vida: un solo modal activo, sidebar oculto durante el detalle y

✿ IDENTIDAD VISUAL

# El sidebar habla el idioma de tu theme

Frame, títulos, nombres, contadores, selección y modal usan colores semánticos del theme seleccionado.

El theme de Pi 1.2 es un Proxy vivo: se evalúa en cada render, así el cambio de tema se refleja al instante.

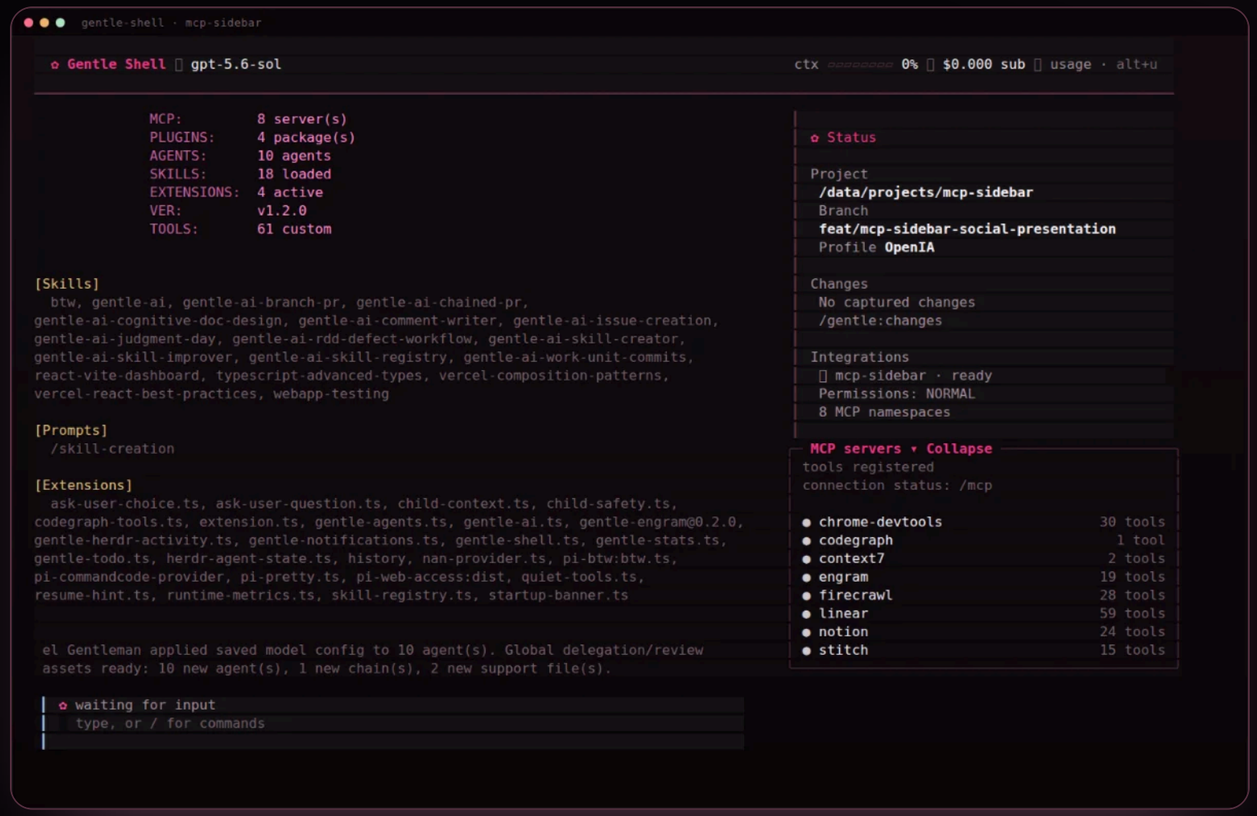
Sin colores congelados en strings: cada frame nuevo consulta el theme actual.

- Prueba independiente de cambio de theme en tiempo de render: los roles semánticos se recomputan sin remount.
- Cobertura unitaria: roles semánticos aplicados a frame, título, cuerpo y selección del sidebar.

✿ EL RAIL COMPARTIDO

# Status, MCP y lo que venga: un solo flujo vertical

- Status queda arriba, MCP debajo, y cualquier panel participante futuro se apila — todo medido por altura renderizada.
- El mouse routing usa offsets reales por panel: el click siempre abre el namespace correcto, aunque `Status` cambie de altura.
- El footer compartido se restaura por identidad al salir, sin pisar a un dueño más nuevo.
- Si el host deja de controlar el layout, MCP desaparece junto con Status — no queda un overlay huérfano.

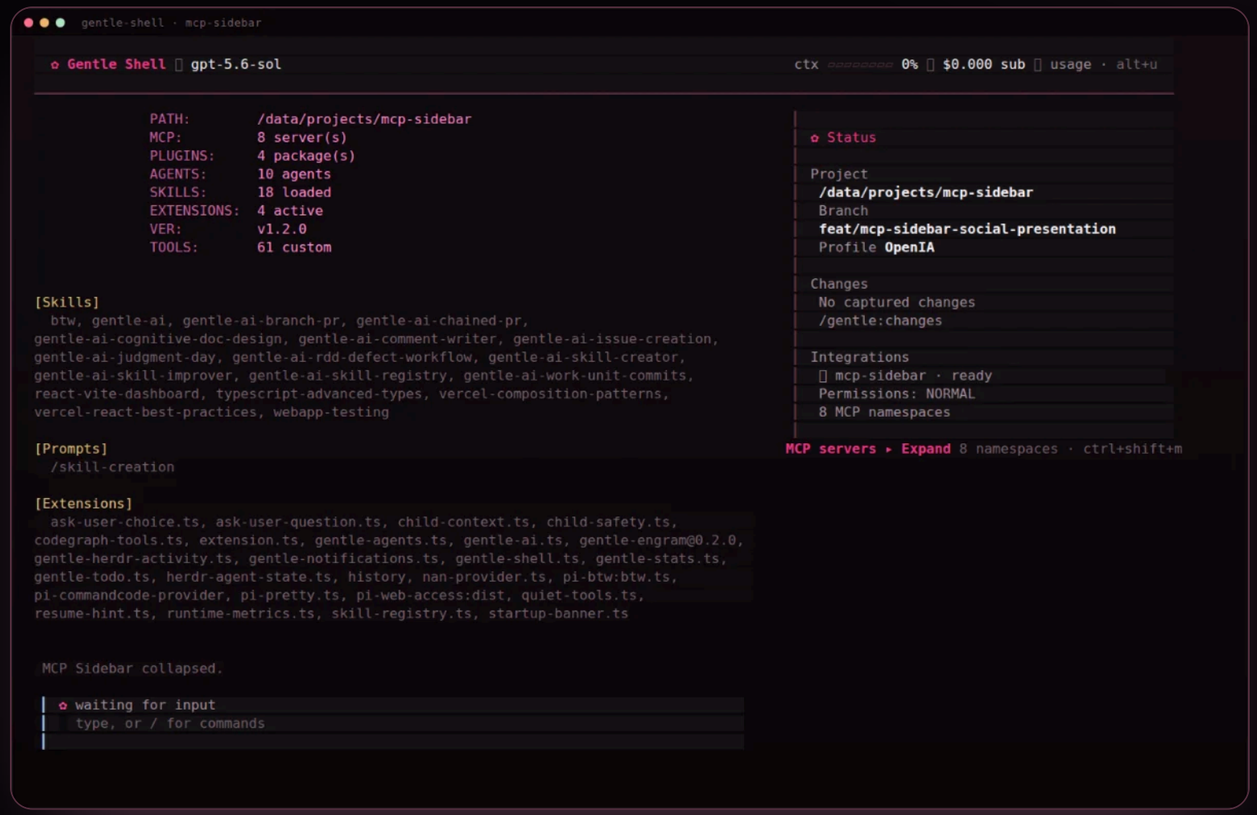



- Prueba real 140→120→140: `Status` y MCP desaparecen juntos y se restauran juntos.
- Los digests de participantes invalidan las cachés de altura de Gentle Shell: nada queda stale.

✿ DETALLE DE PRODUCTO

# Colapsable como Todo, invisible como debe ser

- Tres formas de colapsar: click en el header, `/mcp-sidebar-collapse` y `Ctrl+Shift+M` donde el terminal lo distinga.
- Colapsado queda una sola fila con estado visible: `MCP servers ▸ Expand 8 namespaces`.
- El estado persiste entre hide/show, refresh, detalles y transiciones rail ↔ overlay.
- Responsive sin sorpresas: cuando el terminal no da más, MCP se esconde con Status y vuelve con él.




- Verificación en vivo 140→120→140: el estado de collapse sobrevive a los cambios de presentación y resize.
- Atajos y comandos probados en Gentle Shell real, incluyendo el click primario en el header.

✿ ARQUITECTURA

# Compatibilidad por detección, nunca por invasión

Feature detection sobre el estado de rail del host: si no existe o es incompatible, el overlay pasivo toma el relevo.

Composición por identidad: el footer original se guarda y se restaura solo si el adapter sigue siendo el dueño.

Contratos separados: `isMounted()` (el wrapper está montado) vs `isActive()` (el host controla el layout) — esta distinción resolvió el bug de visibilidad angosta.

- El bug más instructivo: el overlay reaparecía a 120 columnas porque `isActive()` mezclaba montaje y control de layout.
- Verificación independiente: S1–S7 PASS y 6 escenarios de fallback confirmados contra baseline.

✿ DISCIPLINA DE INGENIERÍA

# Test-first, evidencia real y verificación independiente

Test-first por contrato: cada regla empieza con un RED que reproduce el bug o la ausencia.

Pruebas reales en Gentle Shell: clicks SGR, atajos de teclado, resize 140→120→140, transiciones de theme.

Verificación independiente de alto riesgo: un segundo agente audita S1–S7 y los escenarios de fallback.

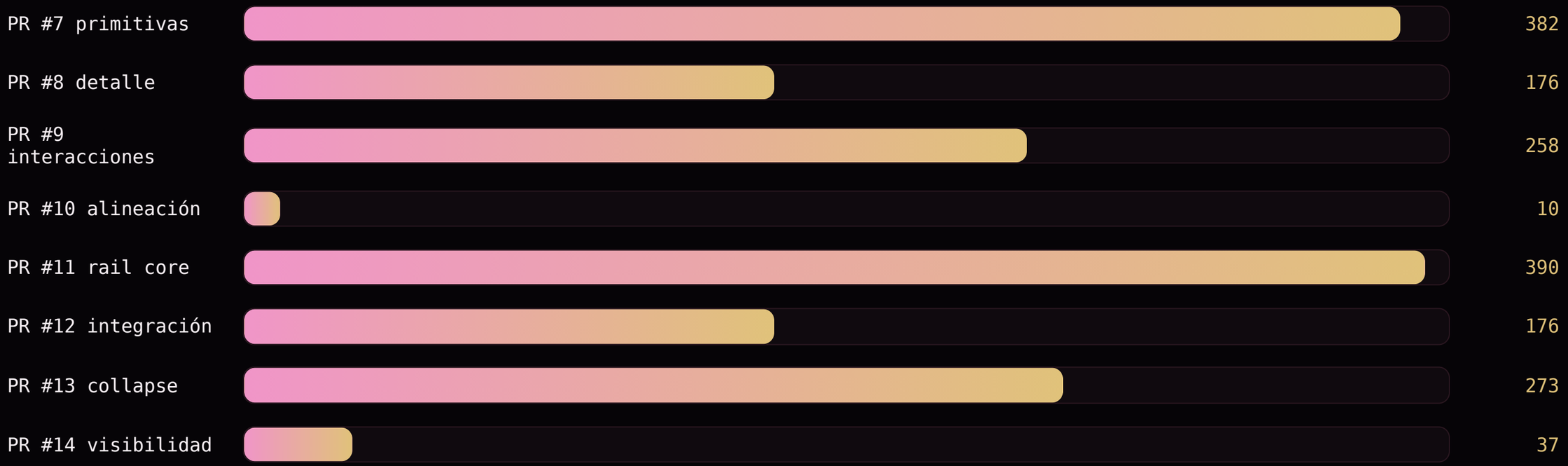
Cierre honesto: `pnpm test`, `pnpm typecheck`, `git diff --check` y `pnpm pack --dry-run` en cada unidad.

- 22/22 tests finales, TypeScript y diff check pasando en el merge.
- El verificador independiente encontró 2 blockers reales en el adapter — corregidos con regresiones antes de cerrar.
- Advisory del verificador corregido: el título expandido excedía el límite de 20 columnas del renderer.

✿ ENTREGA

# 1.670 líneas, 8 PRs y una revisión amigable

Líneas cambiadas por slice · presupuesto de revisión: 400




- Diff total de 1.670 líneas dividido en slices de 382 · 176 · 258 · 10 · 390 · 176 · 273 · 37 líneas.
- PRs #7 a #14 + tracker #6, todos MERGED con el árbol final verificado contra el source.

✿ RESULTADO

# Merged, verificado y listo para usar

Un paquete Pi: sidebar pasivo + rail nativo + detalles interactivos + collapse + theme + responsive.

Funciona en Gentle Shell completo y degrada elegantemente en Pi standalone.

Todo el ciclo documentado: specs, logs de decisión, evidencia de verificación y PRs revisables.

- PR #6 (tracker) merged en `main` como `81ef2eb` — 9 PRs en total, todos MERGED.
- 22/22 tests, typecheck, diff check y package dry-run: verdes en el merge final.

✿ PRÓXIMO PASO

# Probalo, leélo y decime qué ves

Código, specs y evidencia: github.com/oniricosistemas/mcp-sidebar

- ¿Usás Gentle Shell o Pi? Probalo y contame qué mejorarías — el roadmap incluye upstream proposals para slots nativos y eventos de estado MCP.
- Lucas M. Sastre · construyendo herramientas para la comunidad de agentes. Hablemos.